Multi-Process Car Parking System

Linux-based Development Project

ENSSAT, Informatique 1

Introduction

This project aims to simulate a car parking system application using a concurrent (multi-tasking) software architecture. Its main goal is to manage the entry and exit of cars in the parking while ensuring safety and efficiency. This document provides an in-depth explanation of the system, including its functionality, program structure, testing procedures, and illustrations.

The implementation uses inter-process communication techniques such as shared memory segments and semaphores for synchronization.

Important Notes

- This is an individual project.
- The pedagogical goals of this project are to reinforce your practice and deepen your understanding of course concepts.
- Not all course concepts need to be implemented. Your implementation choices may require omitting some concepts (e.g., process synchronization may use semaphores, signals, or pipe waiting).
- Read each section in full before implementing it to understand its overall objective. Then, complete the tasks sequentially.
- Do not use any code generator (Jules, Claude, Copilot, etc.). The resulting code of the project will not be evaluated. You'll be asked questions about your understanding of the project during the final evaluation.

Contents

Overview of the application **3**
March: Basic Version of the Application 6
Task 1 - Structures 6
Task 2 - Basic implementation 6
Code Organization 6
Running instructions 8
Task 3 - Signal Handler for the server 9
Task 4 - Secure the access to the shared memory segment 9
Task 5 - Add the log monitoring implemented by another team 9
Task 6 - Securize the accesses to the log file 10

May: Dealing with logger module updates **11**
Task 7 - Handling logger failures 11
Task 8 - Proof of Concept (POC) of the thread version for logger failure . . . 12

Task 9 - Comparison multi-process vs. multi-thread **12**
Task 10 (optional) - Leveraging AI-based Code Generators 12
Task 11 (optional) - Documenting the code of the application (optional for the project but mandatory in real life!) 13

Task 12 - Enssat Delivrable **16**

Overview of the application

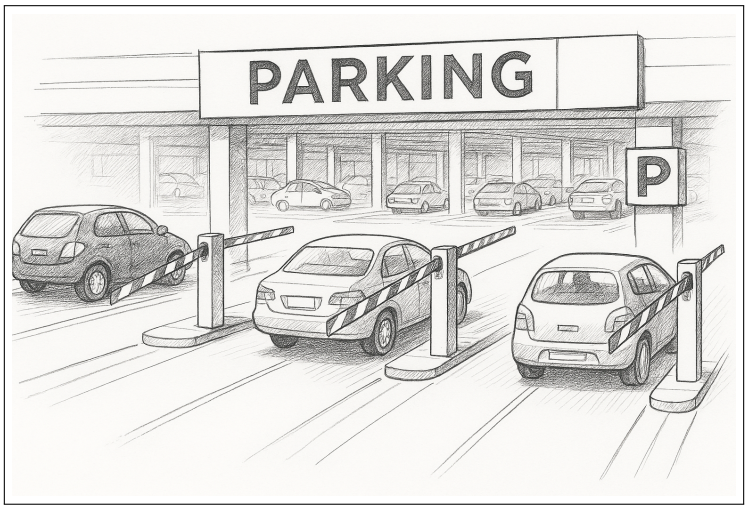

Image generated by AI using Microsoft Copilot.

The car parking system application simulates a multi-process environment in which a central server manages parking resources, and multiple client processes represent individual cars interacting with the system. The architecture follows a client-server model with clear separation of responsibilities. The application uses a shared memory segment for storing the state of the parking.

The server process acts as a central parking manager, responsible for:

- maintaining the state of the parking spots (number of occupied/free spots),
- managing shared resources through inter-process communication mechanisms,
- handling concurrent requests from multiple client processes,
- implementing synchronization mechanisms to prevent race conditions,
- processing entry and exit requests from cars, and
- logging its activities for monitoring and debugging purposes.

The server also actively prevents rule violations by checking capacity before allowing new entries in the parking (no more cars can enter when the parking is full), and maintaining accurate state information (only cars that have entered can exit).

Each client process represents an individual car request, which interacts with the parking system by requesting entry into the parking when arriving, or requesting exit from the parking when leaving. Each client waits for server confirmation to know if it can enter (or exit). Each client logs its activities for monitoring and debugging purposes.

The clients and the server communicate through communication channels provided by the Linux OS kernel.

Architecture of the Basic application

The parking state is stored in a shared memory. The server process is the only process that accesses this memory (we recall that the server process is responsible for handling clients' requests and maintaining the state of the parking spots).

A client process is created when a request is made by a car to enter or exit the parking. A command can have the following form.

```
./client 007 enter
```

Such a command execution creates a client process that sends a request modelling the fact that car number `007` wants to enter the parking.

When a car (client) wants to enter the parking lot, it sends an entry request to the server; this is illustrated in Stage (1) of Figure 1. Upon receiving the request, the server checks the shared memory to determine if any parking spots are available. If a spot is free, then the server updates the shared memory to reflect the new parking state; this corresponds to Stage (2) in Figure 1. Finally, the server sends a response to the client indicating whether the car is allowed to enter, as shown in Stage (3) of Figure 1.

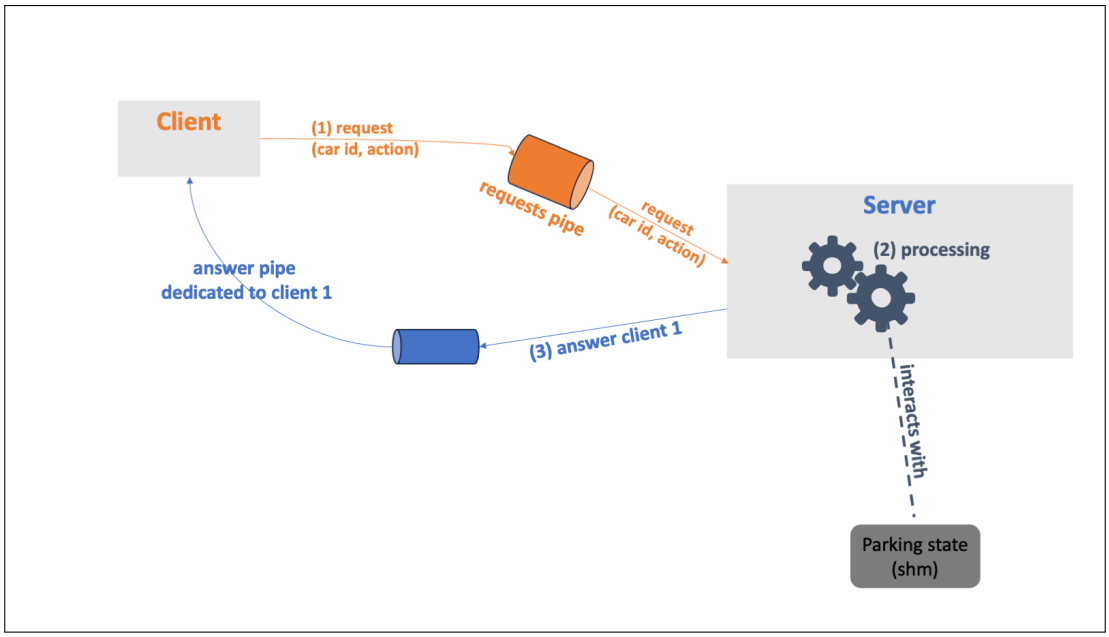

Figure 1: Communication between a client and the server

Similarly, when a car (client) intends to exit the parking lot, it sends an exit request to the server. Upon receiving the request, the server updates the shared memory to reflect the car's departure and the newly available parking spot. This process ensures the parking status remains accurate. As an illustration, the command representing the exit request for car number 124 takes the following form.

```
./client 124 exit
```

Multiple clients can send requests concurrently. In this case, the system architecture corresponds to Figure 2. As shown, all clients transmit their requests to the server through a common communication channel: the *requests pipe* in Figure 2. However, the server replies to each client via a dedicated response channel: one distinct answer channel per client (the *answer pipes* that appear in Figure 2). The server processes requests sequentially, handling only one at a time.

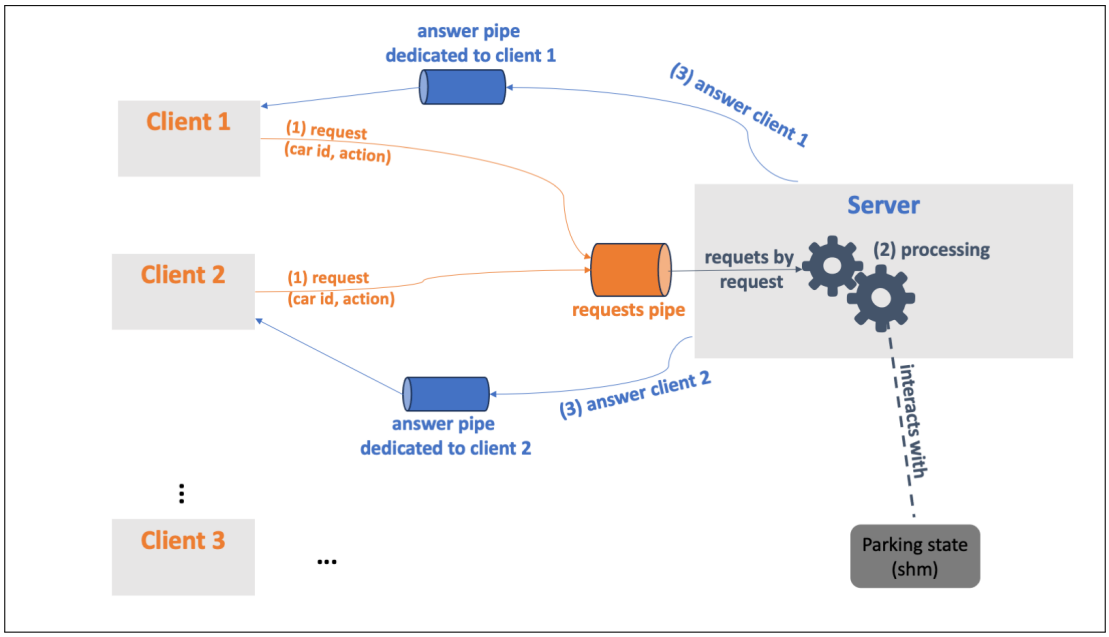

Figure 2: Architecture of the application implying several clients

Important – Project Submission

- The project code itself is **not graded directly**; however, **one or more exam questions** will be drawn from the project, so understanding of your implementation is essential.
- At the end of the last project session, you must **deposit your project on Moodle** so that the teacher can review your work (there is a specific deposit on Moodle).
- All functions, code, and comments must be **written in English**.
- You are encouraged to use **GitHub** to develop your project.

March: Basic Version of the Application

We're in March. You must develop a first version of the system, called the *Basic version*.

Task 1 - Structures

Data structures needed for the project are given in the file *structure.h* (you can download it from the ENT[1]).

- If you're stemming from MPI, you do not download the `structure.c` file and code it yourself according to the `structure.h` file.
- Otherwise (if you're not stemming from MP), you download the two files *structure.h* and *structure.c*

Now you've got two files in your project : *structure.h* and *structure.c*

```
Project
└── Basic Version
    ├── structure.h (new file downloaded from ENT)
    └── structure.c (new file downloaded from ENT if not MPI, or implemented if
        MPI)
```

Task 2 - Basic implementation

We now turn our attention to the practical implementation of the car parking system. The application follows a modular architecture, with code organized across multiple files to ensure maintainability, scalability, and clear separation of concerns.

Code Organization

The system is implemented using a collection of specialized modules, each responsible for distinct functionalities, described below.

`structures.h`: Shared data structures (from Task 1).

`structures.c`: Additional function for data structures (from Task 1).

`inspector.c`: An independent module that can print the parking state (meaning that `./inspector` can be executed during the other processes of the application). To make the project easier, a first version of the file *inspector.c* is provided to you: download it from the ENT.

`logger.h`: Download it, but do not use it for the moment.

`logger.o`: Download it, but do not use it for the moment.

`server.c`: Core server implementation (not provided, its implementation must be completed as part of Task 2).

[1] `https://foad.univ-rennes.fr/course/view.php?id=1008362`

client.c: Client-side logic for car operations and user interaction (not provided, its implementation must be completed as part of Task 2).

Makefile: A makefile that makes it easier to compile the project: download it from the ENT. The makefile may need adjustments according to your system (for instance, the mylogger is not implemented for the moment, you should have to delete it from the Makefile). If you do not want to use it, you can compile the files yourself (see the Running Instructions in the following).

This modular design enables a clear separation of system responsibilities, independent development and testing of components, easier maintenance and future enhancements, and improved debugging through component isolation.

After downloading the provided material, your project is in the following state before you proceed with Task 2.

```
Project
└── Basic Version
    ├── structure.h (downloaded from ENT)
    ├── structure.c (downloaded from ENT)
    ├── inspector.c (new file downloaded from ENT)
    ├── logger.h (new file downloaded from ENT, but do not use it for the moment)
    ├── logger.o (new file downloaded from ENT, but do not use it for the moment)
    └── Makefile (new file downloaded from ENT)
```

Task 2 consists in implementing the *server.c* and the *client.c* files.

After implementing Task 2, your project should have the following organization.

```
Project
└── Basic Version
    ├── structure.h (downloaded from ENT)
    ├── structure.c (downloaded from ENT)
    ├── inspector.c (downloaded from ENT)
    ├── logger.h (download it, but do not use it for the moment)
    ├── logger.o (download it, but do not use it for the moment)
    ├── client.c (<- !! new file, this is your job for Task 2)
    ├── server.c (<- !! new file, this is your job for Task 2)
    └── Makefile (downloaded from ENT)
```

Running instructions

To run the program, you can follow the following steps.

1. Compile all source files using GCC with appropriate flags (the flags may differ according to your OS installation).

```
# Cleaning
$ rm structures.o server.o client.o inspector.o server client inspector *.log
# Compile server
$ gcc -Wall -Wextra -o server server.c structures.c mylogger.c logger.o
# Compile client
$ gcc -Wall -Wextra -o client client.c structures.c mylogger.c logger.o
# Compile inspector
$ gcc -Wall -Wextra -o inspector inspector.c structures.c mylogger.c logger.o
```

   or

```
$ make clean ; make
```

   (The Makefile may need to be adjusted as you progress through the different stages of the project.)

2. Run the server in one terminal with the parking capacity as an argument:

```
# Terminal 1
$ ./server 10
```

3. Run multiple client instances in other terminals:

```
# Terminal 2
$ ./client 007 enter
```

```
# Terminal 3
$ ./client 42 enter
```

```
# Terminal 4
$ ./client 007 exit
```

```
# Terminal 1: Two cars arriving simultaneously
$ ./client 002 enter & ./client 023 enter
```

Task 3 - Signal Handler for the server

Implement a SIGINT (ctrl-c) signal handler for the server that ensures graceful termination. The handler must, in particular, properly release any open semaphores.

Task 4 - Secure the access to the shared memory segment

When both the `inspector` and `server` processes attempt to access the shared memory segment concurrently, several critical synchronization issues arise. Most significantly, the inspector may read and display inconsistent or partially updated parking states, leading to misleading monitoring information. To prevent these race conditions and ensure data integrity, we must implement proper synchronization using the semaphore `SHM_PARKING_STATE`.
This semaphore serves as a mutual exclusion mechanism, guaranteeing that only one process can access the shared memory at any given time. To locate its declaration in the codebase, you can use the `grep` command.

After implementing Task 4, your project has the same organization, with new versions of *inspector.c* and *server.c*

```
Project
└── Basic Version
    ├── structure.h (downloaded from ENT)
    ├── structure.c (downloaded from ENT)
    ├── inspector.c (<- !! new version)
    ├── logger.o (download it, but do not use it for the moment)
    ├── client.c (from Task 2)
    ├── server.c (<- !! new version)
    └── Makefile (downloaded from ENT)
```

Test your code. If needed, introduce artificial delays in critical sections using `sleep()` to observe the semaphore behavior clearly.

Task 5 - Add the log monitoring implemented by another team

Now you need to add log monitoring functionality to trace both client requests and server processing. The logger functions have been implemented by another of the project. The logger provided by the other team is composed of a file *logger.h* (the interface) and a file *logger.o* (it's a pre-compiled file), that you have already downloaded. The logger provides a procedure called `log_message` (see its signature in the *logger.h* file) that allows writing a log in a log file whose name is defined by the logger. **Note that the logger module was developed by another team and cannot be modified.**

According to the provided logger, all processes -both clients and servers- write their logs to a shared log file. To demonstrate this functionality, draw yourself in Figure 2 the log file and its connections made by the processes. (There is no hidden complexity; completing the figure is straightforward.)

In Task 5, you must implement logging for both the server and the client processes by using the logger module. For debugging purposes, the logger captures client requests, car entry/exit operations, and system messages (errors, warnings, and information), from the client's point of view, and from the server's point of view. Find the log file created by the logger, and check its content according to the usage that you made of the logger.

After implementing Task 4, your project has the same organization, with new versions of *inspector.c* and *server.c*

```
Project
└── Basic Version
    ├── structure.h
    ├── structure.c
    ├── inspector.c
    ├── logger.o
    ├── client.c (<- !! new version with logs)
    ├── server.c (<- !! new version with logs)
    └── Makefile
```

Test your program again after implementing the logging functionality.

Completion: Congratulations! You've completed the "Basic version" of the application. Save this version as the Basic version.

Task 6 - Securize the accesses to the log file

You do not see it, but the current logger lacks access control mechanisms for the log file. Implement a secure solution by creating a new module called `mylogger` that wraps the existing logger functionality. This module should use a semaphore initialized to 1 to ensure atomic write operations. The implementation must follow a strict pattern: acquire the semaphore before writing, perform the logging operation by using the `log_message` procedure of the logger, then immediately release the semaphore. This will prevent race conditions during concurrent access while maintaining all existing logging capabilities.

After implementing Task 6, your project has the following organization:

```
Project
└── Basic Version
    ├── structure.h
    ├── structure.c
    ├── inspector.c
    ├── logger.o
    ├── mylogger.c (<- !! new file)
    ├── mylogger.h (<- !! new file)
    ├── client.c (<- !! new version with securized access to the log file)
    ├── server.c (<- !! new version with securized access to the log file)
    └── Makefile
```

May: Dealing with logger module updates

The logger team has produced a new version after two months of work. Download it (from the ENT).
Your project has the following organization:

```
Project
└── NewLogger
    ├── structure.h
    ├── structure.c
    ├── inspector.c
    ├── mylogger.o (<- !! new version, which handled logger failures)
    ├── mylogger.c
    ├── mylogger.h
    ├── client.c
    ├── server.c
    └── Makefile
```

Test this new version. You should find that the logger can fail (verify it).

You cannot modify the logger, but must make your code robust, which is your Task 7.

Task 7 - Handling logger failures

You must handle logger failures by delegating each log writing to a child process and capturing the failures.

After implementing Task 7, your project has the following organization:

```
Project
└── Basic Version
    ├── structure.h
    ├── structure.c
    ├── inspector.c
    ├── logger.o
    ├── mylogger.c (<- !! new version, which handled logger failures)
    ├── mylogger.h
    ├── client.c
    ├── server.c
    └── Makefile
```

Completion: Congratulations! You've completed the "NewLogger version" of the application. Save this version as the Basic version.

July: Multi-thread Version

Your boss wants to explore whether a multi-threaded version would be more efficient for handling the logger.

Task 8 - Proof of Concept (POC) of the thread version for logger failure

You must implement a version that delegates the logger to a thread.
Your POC has the following organization:

```
Project
└── POC_Thread4LoggerFailures
    ├── structure.h
    ├── structure.c
    ├── inspector.c
    ├── mylogger.o (<- !! new version, which handled logger failures by a thread)
    ├── mylogger.c
    ├── mylogger.h
    ├── client.c
    ├── server.c
    └── Makefile
```

Test this version. What do you conclude?

Task 9 - Comparison multi-process vs. multi-thread

Based on this experience and the experience you had in practical sessions, prepare a short presentation for your boss with your conclusions (pros and cons of both approaches in terms of life cycle, communication, ease of programming, security, ...)

Task 10 (optional) - Leveraging AI-based Code Generators

In this final phase of the project coding, you will engage with an AI-based code generator such as Jules, Copilot, Claude, ... to implement the parking system application. Begin by requesting the AI to generate the complete implementation of your project. Once the code is produced, conduct thorough testing to verify its functionality and correctness. Throughout this process, maintain an active dialogue with the AI to refine the implementation and ensure it meets all project requirements. Take the time to carefully analyze the AI's proposals, understanding both the solutions it suggests and any limitations they may have. Work iteratively with the AI to guide the implementation toward your specific project goals. After achieving a working version, perform comprehensive final testing to validate the complete solution. Finally, document your experience with using AI for this project, including observations about the AI's capabilities, limitations, and how effectively it could be integrated into the development process.

Spoiler alert...

> *Vibe coding* — the practice of developing software through iteratively conversing with a large language model (LLM) — has exploded in popularity

within the last year. However, developers report key limitations including the accumulation of technical debt, security issues, and code churn to achieve satisfactory results. We argue that these pitfalls result from LLMs' inability to reconcile accumulating human-imposed constraints during vibe coding, with developers inadvertently failing to resolve contradictions because LLMs prioritize user commands over code consistency.

Jacqueline Mitchell and Yasser Shaaban. 2025. Position: Vibe Coding Needs Vibe Reasoning: Improving Vibe Coding with Formal Verification. In Proceedings of the 1st ACM SIGPLAN International Workshop on Language Models and Programming Languages (LMPL '25). Association for Computing Machinery, New York, NY, USA, 84–90. `https://doi.org/10.1145/3759425.3763390`.

Task 11 (optional) - Documenting the code of the application (optional for the project but mandatory in real life!)

It should have been done all along the project but you'll do it in a dédicated task (Task 11). Good documentation is an integral part of software quality. In this task, you are required to document your source code using **Doxygen**, a widely used documentation generator that produces HTML (and LaTeX) references directly from annotated source comments.

What is Doxygen?

Doxygen parses your source files, extracts specially formatted comments, and generates a browsable HTML documentation website (or a PDF) with no extra effort on your part. A live preview of the kind of output Doxygen produces can be found at `https://www.doxygen.nl/results.html`.

Basic usage

1 – Installation. If you use your own computer, then you have to install Doxygen.

- **Linux / macOS:** `sudo apt install doxygen` / `brew install doxygen`
- **Windows:** download the installer from `https://www.doxygen.nl/download.html`.

2 – Generate a default configuration file. Run the following command once, at the root of your project:

```
doxygen -g Doxyfile
```

Open `Doxyfile` and set at least:

- `PROJECT_NAME` – the name of your project,
- `INPUT` – the path(s) to your source files,
- `RECURSIVE = YES` – to scan sub-directories,
- `EXTRACT_ALL = YES` – to document every symbol, even those without a comment block.

3 – Build the documentation.

```
doxygen Doxyfile
```

The HTML output is placed in `html/index.html` by default.

Comment syntax

Doxygen recognises *special comment blocks* that start with `/**` (Javadoc style) or `/*!` (Qt style). The most common tags are listed in Table 1.

Tag	Purpose
`@brief`	One-line summary of the element
`@param`	Description of a function parameter
`@return`	Description of the return value
`@note`	Additional note or warning
`@see`	Cross-reference to another element
`@file`	Documents the file itself
`@author`	Author of the file / module

Table 1: Commonly used Doxygen tags.

Annotated example

The listing below shows a minimal but complete example of a documented header file.

```c
/**
 * @file stack.h
 * @brief A simple integer stack.
 * @author Alice Martin
 */

#ifndef STACK_H
#define STACK_H

/** Maximum number of elements the stack can hold. */
#define STACK_MAX 128

/**
 * @brief Stack data structure.
 *
 * Stores up to STACK_MAX integers in LIFO order.
 */
typedef struct {
    int data[STACK_MAX]; /**< Internal storage array. */
    int top; /**< Index of the top element. */
} Stack;

/**
 * @brief Push an integer onto the stack.
 *
```

```c
 * @param s Pointer to the stack.
 * @param val Value to push.
 * @return 1 on success, 0 if the stack is full.
 */
int stack_push(Stack *s, int val);

/**
 * @brief Pop the top integer from the stack.
 *
 * @param s Pointer to the stack.
 * @param out Pointer to store the popped value.
 * @return 1 on success, 0 if the stack is empty.
 * @note The value at *out is undefined if the stack is empty.
 */
int stack_pop(Stack *s, int *out);

#endif /* STACK_H */
```

Listing 1: Example of a Doxygen-annotated header.

Task

1. Add Doxygen comment blocks to **every** function, structure, macro, and file in your project. At minimum, each function must have a `@brief`, one `@param` per parameter, and a `@return` tag (where applicable).
2. Generate the HTML documentation with `doxygen Doxyfile` and verify that no warnings are emitted.
3. Include the generated `html/` folder in your submission archive **and** provide a `README.md` explaining how to regenerate it.

Grading criteria.

- **Completeness** – all public symbols are documented.
- **Accuracy** – descriptions match the actual behaviour.
- **Clarity** – comments are written in clear English.
- **Output** – the HTML documentation builds without errors.

Task 12 - Enssat Delivrable

Important – Project Submission

- The project code itself is **not graded directly**; however, **one or more exam questions** will be drawn from the project, so understanding of your implementation is essential.
- At the end of the last project session, you must **deposit your project on Moodle** so that the teacher can review your work (there is a specific deposit on Moodle).